字符串

串串

Rosaya

Oasis

目录

1. 基本概念 2
1.1 定义 3
1.2 哈希 4
1.3 扩展 9
1.4 习题 10
2. 字典树 11
2.1 例题 12
2.2 定义 13
2.3 扩展 15
2.4 习题 16
3. KMP 算法 17

目录

1. 基本概念 2
1.1 定义 3
1.2 哈希 4
1.3 扩展 9
1.4 习题 10
2. 字典树 11
2.1 例题 12
2.2 定义 13
2.3 扩展 15
2.4 习题 16
3. KMP 算法 17

1.1 定义

1.1 定义

- 字符集：一个**字符集** Σ 是一个建立了全序关系的集合，也就是说有且仅有一种方式将 Σ 中所有元素按照升序排序。
- 字符串：一个**字符串** S 是 n 个字符顺次排列形成的序列，记 $|S| = n$ 表示 S 的长度。记 S_i 表示 S 的第 i 个字符（下标从 1 开始）。
- 子串：S 的**子串** $S[i,j](i \le j)$ 表示顺次排列 $S_i, S_{i+1}, ..., S_j$ 得到的字符串。
- 子序列：S 的**子序列** S' 表示选出若干下标 $1 \le p_1 \le p_2 \le ... \le p_k \le |S|$ 后顺次排列 $S_{p_1}, S_{p_2}, ..., S_{p_k}$ 得到的字符串，也就是说选择的位置相对顺序不变。
- 前缀 & 后缀：S 的**前缀** $\mathrm{Pre}_S(i)$ 表示 $S[1,i]$，**后缀** $\mathrm{Suf}_S(i)$ 表示 $S[i,|S]$。特别地，若 $i \ne |S|$，则称 $\mathrm{Pre}_S(i)$ 为 S 的**真前缀**，若 $i \ne 1$，则称 $\mathrm{Suf}_S(i)$ 为 S 的**真后缀**。
- 字典序：两个**不同**的字符串 S, T 有 $S < T$ 当且仅当：S 是 T 的某个前缀，或存在一个 i 使得 $\mathrm{Pre}_S(i) = \mathrm{Pre}_T(i)$ 且 $S_{i+1} < T_{i+1}$。

1.2 哈希

1.2 哈希

P3370【模板】字符串哈希

给定 n 个字符串 $S_1, S_2, ..., S_n$，求出 n 个字符串中有多少个不同的字符串。

$n \le 10000$，$|S_i| \le 1500$。

1.2 哈希

P3370【模板】字符串哈希

给定 n 个字符串 $S_1, S_2, ..., S_n$，求出 n 个字符串中有多少个不同的字符串。

$n \le 10000$，$|S_i| \le 1500$。

字符串比对

给定 n 个字符串 $S_1, S_2, ..., S_n$。

T 组询问，每次给定 a, b, i, j, k，回答 $S_{a[i,i+k-1]}$ 和 $S_{b[j,j+k-1]}$ 是否相同。

$n, T \le 10^5$，$\sum |S_i| \le 10^6$。

1.2 哈希

1.2 哈希

我们发现比较两个字符串的复杂度是 $O(|S|)$ 的，其中 $|S|$ 是这两个字符串的长度。

如果有什么方法能够**快速判断**两个字符串是否相等就好了。

1.2 哈希

我们发现比较两个字符串的复杂度是 $O(|S|)$ 的，其中 $|S|$ 是这两个字符串的长度。

如果有什么方法能够**快速判断**两个字符串是否相等就好了。

我们考虑能否以一种方式压缩字符串信息，能够使**本来不同的字符串**压缩后的信息大概率不相同，而**本来相同的字符串**压缩后的信息相同。

基于以上想法，我们可以使用**哈希**来解决此类问题。

1.2 哈希

1.2 哈希

哈希，也称为 Hash，指我们构造一个把**字符串**映射到**整数**的函数 f，将其称为 Hash **函数**。

那么此时如果有 $f(S) = f(T)$，我们就认为 $S = T$，这样比较两个字符串是否相等就转换为了比较函数值是否相等，复杂度由 $O(|S|)$ 降至了 $O(1)$。

现在一个新的问题出现了：如何构造 f。

一般而言，我们会选取一个比较大的 base，然后令 $f(S) = \left(\sum_{i=1}^{|S|} S_i \cdot \text{base}^{|S|-i}\right) \bmod M$，这样 $O(|S|)$ 即可预处理出一个字符串的哈希值，可以理解为把 S 转换成了一个 base 进制数然后对一个 M 取模。

于是第一题可以在 $O(\sum |S_i| + n^2)$ 的复杂度内解决。

1.2 哈希

1.2 哈希

考虑 $f(S) = \left(\sum_{i=1}^{|S|} S_i \cdot \text{base}^{|S|-i}\right) \bmod M$ 有什么要求。

首先就是 S_i 的值不能为 0（但当我们使用 ASCII 码时就不会出现这个问题）。

其次就是 $S_i < \text{base}$，否则这个 base 进制数并不能唯一对应回一个字符串 S。

再者是如果存在整数 k 使得 $M \mid \text{base}^k$，那么相当于取模后只保留了字符串后 k 位，所以一般要求 $(\text{base}, M) = 1$，也就是它们互质。

最后一点，M 不能过大或过小，一般选取为 $10^9 + 7, 2^{64}$，对应常用的 base 为 $131, 1331, 13331$。

这样，如果我们认为 $f(S)$ 分布足够均匀，那么我们可以认为 $S \neq T, f(S) = f(T)$ 出现的概率很小。

如果冲突的概率有些大怎么办？可以再选取一组基 base' 和模数 M' 进行**双哈希验证**！

1.2 哈希

考虑如何求出 $S[l,r]$ 的哈希值 $f(S[l,r])$。

从 f 的构造出发，$f(S[l,r]) = \left(\sum_{i=l}^{r} S_i \cdot \mathrm{base}^{r-i}\right) \bmod M$，如果我们已知 $f(S[l,r])$，那我们可以 $O(1)$ 算出 $f(S[l-1,r])$ 和 $f(S[l,r+1])$。

也就是说，我们可以在 $O(|S|)$ 的复杂度内求出 $f(S[1,i])$。

然后我们就能推导得出 $f(S[l,r]) = \left(f(S[1,r]) - \mathrm{base}^{r-l+1} \cdot f(S[1,l-1])\right) \bmod M$。

于是第二题可以在 $O(\sum |S_i| + T)$ 的复杂度内解决。

1.3 扩展

1.3 扩展

1. 最长回文子串

2. 允许 k 次失配的字符串匹配

3. 最长公共子字符串

1.3 扩展

1. 最长回文子串

2. 允许 k 次失配的字符串匹配

3. 最长公共子字符串

可以使用**二分哈希**求出两个字符串 S, T 的**最长公共前缀**，从而解决上述问题。

此外，用二分哈希的方式书写字符串的比较函数，即可用 sort 给若干字符串排序，复杂度 $O(n \log^2 n)$。

二分哈希在字符串算法中是及其重要的，因为它甚至可以仅仅以稍劣一些的复杂度完美替换 KMP，Manacher，甚至是后缀数组。

除了字符串哈希，还有集合哈希，序列哈希，树哈希等等，处理思路则是全部转化为**序列哈希**（字符串哈希可以直接视为一种序列哈希）。

1.4 习题

- **CF113B Petr#**
- **P2312 [NOIP2014 提高组] 解方程**
- **CF985F Isomorphic Strings**
- **P8819 [CSP-S 2022] 星战**

目录

1. 基本概念 2
1.1 定义 3
1.2 哈希 4
1.3 扩展 9
1.4 习题 10
2. 字典树 11
2.1 例题 12
2.2 定义 13
2.3 扩展 15
2.4 习题 16
3. KMP 算法 17

2.1 例题

2.1 例题

P8306【模板】字典树

给定 n 个字符串 $s_1, s_2, \ldots, s_n$ 和询问次数 q，第 i 次询问给出一个字符串 t_i 求 t_i 是多少个 s_j 的前缀。

$n, q \le 3 \times 10^5$，$\sum |s_i| + \sum |t_i| \le 3 \times 10^6$。

2.1 例题

P8306【模板】字典树

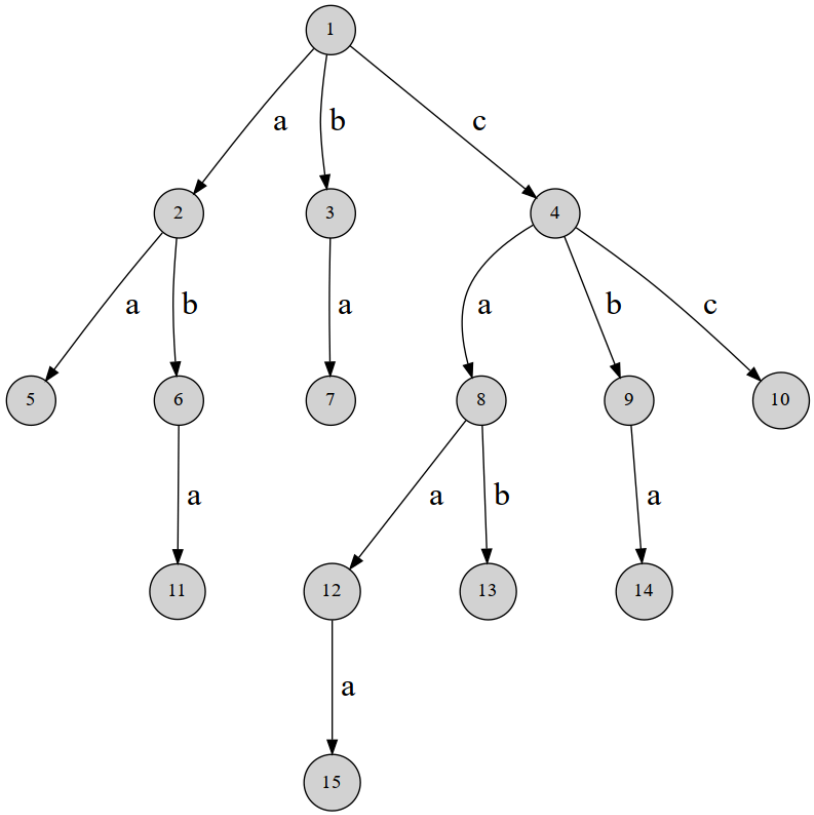
给定 n 个字符串 $s_1, s_2, \ldots, s_n$ 和询问次数 q，第 i 次询问给出一个字符串 t_i 求 t_i 是多少个 s_j 的前缀。

$n, q \le 3 \times 10^5$，$\sum |s_i| + \sum |t_i| \le 3 \times 10^6$。

n 个模式串的信息难以**有效整合**，目前我们只能一一比对哈希值，复杂度为 $O(nq)$。

2.2 定义

2.2 定义



Trie

2.2 定义

2.2 定义

定义**字典树**（Trie 树）是一类边、点均存有信息的有根树。

其中有向边上存储了一个字符，点上存储了这个点是否为某个模式串的终止节点。

对于任意点 u，根 rt 到 u 的路径上的字符**顺序排列**可以构成一个模式串的**前缀**，特别地，如果 u 上有终止标记，那么 rt 到 u 的路径上的字符可以构成一个模式串。

2.2 定义

定义**字典树**（Trie 树）是一类边、点均存有信息的有根树。

其中有向边上存储了一个字符，点上存储了这个点是否为某个模式串的终止节点。

对于任意点 u，根 rt 到 u 的路径上的字符**顺序排列**可以构成一个模式串的**前缀**，特别地，如果 u 上有终止标记，那么 rt 到 u 的路径上的字符可以构成一个模式串。

得到这些信息后，我们可以快速统计对于每个节点 u 对应的字符串，有多少模式串以它为前缀。

其实也就是 u 子树内**终止标记的数量**。

事实上，绝大多数**前缀相关**的信息都可以用字典树储存，故其也被称为前缀树。

这样例题可以在 $O(\sum|s_i| + \sum|t_i|)$ 时间内解决。

2.3 扩展

2.3 扩展

事实上字典树的应用范围也很广。

例如储存二进制数的 01-Trie，就很广泛的应用于一系列**按位贪心**中。

2.3 扩展

事实上字典树的应用范围也很广。

例如储存二进制数的 01-Trie，就很广泛的应用于一系列**按位贪心**中。

P4551 最长异或路径

给定一棵 n 个点的带权树，结点下标从 1 开始到 n。寻找树中找两个结点，求最长的异或路径。

异或路径指的是指两个结点之间唯一路径上的所有边权的异或。

$n \leq 10^5$，$0 \leq w < 2^{31}$。

2.3 扩展

事实上字典树的应用范围也很广。

例如储存二进制数的 01-Trie，就很广泛的应用于一系列**按位贪心**中。

P4551 最长异或路径

给定一棵 n 个点的带权树，结点下标从 1 开始到 n。寻找树中找两个结点，求最长的异或路径。

异或路径指的是指两个结点之间唯一路径上的所有边权的异或。

$n \le 10^5$，$0 \le w < 2^{31}$。

只需要进行一步转化：$c(u,v) = c(1,u) \oplus c(1,v)$，即可按位贪心求解，其中 $c(u,v)$ 是 u,v 路径上所有边的异或和。

2.4 习题

- **P2580 于是他错误的点名开始了**
- **P9753 [CSP-S 2023] 消消乐**

目录

1. 基本概念 2
1.1 定义 3
1.2 哈希 4
1.3 扩展 9
1.4 习题 10
2. 字典树 11
2.1 例题 12
2.2 定义 13
2.3 扩展 15
2.4 习题 16
3. KMP 算法 17